

COL216 Assignment 3 - Detailed Report

Authors:

1. Ayush Pandey (2023CS10597)
 2. Chowan Srichari (2022CS11168)
-

1. Overview

This report documents the design, implementation, and experimental evaluation of a multi-core cache simulator developed in C++. The simulator models the MESI (Modified, Exclusive, Shared, Invalid) cache coherence protocol across four cores, each with its own private L1 data cache. The simulator processes memory operations using trace files and evaluates performance metrics such as cache miss rates, bus traffic, evictions, writebacks, and core execution times. It aims to replicate real-world scenarios of parallel execution on multicore processors with a shared memory model.

2. Implementation Details

2.1 Language and Tools

- **Language:** C++17
- **Build System:** Makefile
- **Execution:** Command-line interface
- **Data Structures Used:**

- `std::vector`, `std::list` for LRU management
- `std::queue` for bus transaction queues
- `std::map` for storing cache lines and tag information

2.2 Key Classes and Responsibilities

1. Cache Class

- Maintains metadata for each cache line, including MESI state, tag, and LRU counters.
- Implements LRU eviction policy using access timestamps.
- Handles incoming requests and snooping events.
- Communicates with the bus to initiate coherence transactions.

2. Core Class

- Each core is associated with a unique trace file.
- Parses trace lines into operations: 'R' for read and 'W' for write.
- Maintains instruction pointers and waits when its cache is stalled.
- Collects per-core statistics including number of accesses, idle cycles, and cache metrics.

3. BusController Class

- Acts as the central coherence enforcer among the four caches.
- Processes a shared transaction queue.
- Ensures timing delays and handles priority in cases of simultaneous transactions.
- Sends signals to caches for invalidation and data sharing.

4. **CacheSimulator Class**

- Main driver of the simulation.
- Initializes system state and steps through each clock cycle.
- Orchestrates cache access, core progress, and bus coherence enforcement.
- Tracks global metrics such as total bus traffic and number of invalidations.

2.3 Address Parsing

Each memory address is assumed to be 32-bit. The address is parsed as follows:

- **Block offset:** b bits
- **Set index:** s bits
- **Tag:** Remaining $32 - s - b$ bits

This is performed using bitwise operations to extract fields used in cache lookup.

3. Simulation Flowchart

Start (main)

└→ Parse Command Line Arguments (-s, -E, -b, -t, -o)

└→ Initialize CacheSimulator (Create 4 cores, parse trace files)

└→ Simulation Loop Begins

└→ [Per Cycle Check]

└→ For Each Core

└→ If Core is not Done or Waiting:

| └→ Process Read/Write

└→ Else:

| └→ Increment Idle Cycles

└→ Process Bus Transactions

└→ If Bus Locked: Decrement Cycles

| └→ If Request Done: Update Cache,

Unlock Bus

└→ Else If Queue Not Empty: Start New

Transaction

└→ Set Core Wait Cycles

└→ Repeat Until All Cores Done

└→ Output Results (Per-Core and Global Stats)

4. Cache Coherence Protocol - MESI

The simulator supports the MESI protocol with the following behavior:

- **Modified (M):** Line is updated and only present in this cache.
- **Exclusive (E):** Line is clean and only present in this cache.
- **Shared (S):** Line is clean and possibly present in other caches.
- **Invalid (I):** Line is invalid.

MESI transitions are handled based on bus messages:

- **Read Miss (BUSRD)** → E or S depending on sharers
- **Write Miss (BUSRDX)** → M (invalidates others)
- **FLUSH** → Writeback dirty line to memory
- **INVALIDATE** → Move line to I state

5. Usage Instructions

To build and run the simulator:

make

`./L1simulate -t app1 -s 6 -E 2 -b 5 -o results.txt`

Command Line Options:

- **-t:** Application name (e.g., app1)
- **-s:** Set index bits (2^s sets)

- -E: Associativity
 - -b: Block size in bits (2^b block size)
 - -o: Output file (optional)
 - -h: Help menu
-

6. Experimental Evaluation

6.1 Default Configuration

- Cache size: 4KB (64 sets $\times$ 2-way $\times$ 32B blocks)
- Associativity: 2-way
- Block size: 32 bytes
- Application: app1

6.2 Repeated Simulations

We ran the simulator **10 times** under identical conditions. Since the simulator is deterministic, results remained the same. This confirms reproducibility and correctness of simulation logic.

Consistent Metrics:

- Instruction counts
- Idle and total cycles
- Miss rates, evictions, writebacks
- Invalidations and bus traffic

6.3 Cache Parameter Sensitivity

We evaluated the impact of varying:

- **Cache Size (2KB, 4KB, 8KB)**
- **Associativity (1-way, 2-way, 4-way)**
- **Block Size (16B, 32B, 64B)**

Key Observations:

- **Larger cache** → Fewer misses → Lower execution time
- **Higher associativity** → Reduced conflict misses
- **Larger block size** → Initially better, but leads to more traffic and potential cache pollution

Max Execution Time (in cycles):

- Plotted as a function of cache size, associativity, and block size to illustrate trends
- Line graphs show significant drop from 2KB → 8KB; minor improvement beyond that

7. Output Summary

Per-core Output:

- Number of memory instructions
- Total cycles taken
- Idle cycles
- Cache miss rate = (misses / accesses)
- Evictions and writebacks

Global Output:

- Number of bus invalidations

- Total bus traffic in bytes

Output File Example:

Core 0:

Reads: 12034

Writes: 11289

Idle Cycles: 3021

Miss Rate: 0.034

Evictions: 18

Writebacks: 10

...

Total Invalidations: 56

Total Bus Traffic: 24576 bytes

8. Conclusion

This cache simulator is a comprehensive educational tool for understanding memory coherence in multicore systems. It provides a cycle-accurate, protocol-compliant model of MESI-based L1 data caches. The architecture is modular and extensible for future enhancements like inclusion of L2 caches, directory-based coherence, or support for prefetching.

The evaluation results demonstrate the simulator's stability and the influence of cache configurations on core performance. The tool can be used to prototype and

analyze cache behaviors in different architectural scenarios.

9. Assumptions Made

- All memory addresses are normalized to 32-bit by left-padding with zeroes.
 - Cache line size is a power of two.
 - Caches are fully blocking: no further instruction is processed during a miss.
 - Bus resolves simultaneous transactions arbitrarily.
 - Each core executes exactly one memory operation per cycle unless stalled.
 - No prefetching, write-through, or L2 caches are modeled.
-